

Implementing Minimum Error Rate Classifier

Dipesh Shome

Department of Computer Science and Engineering, AUST

Ahsanullah University of Science and Technology

Dhaka, Bangladesh

160204045@aust.edu

Abstract—In this experiment, I tried to implement Minimum error rate classifier using the posterior probabilities which uses Normal distribution to calculate likelihood probabilities to classify given sample points. The objective of minimum error rate classifier is to minimize the error rate during classification. The implementation is followed by some steps: calculate likelihood using normal distribution then make decision rule to classify sample points and then draw decision boundary. This classifier is also known as Byes classifier with minimum error rate.

Index Terms—minimum error rate classifier, Bayesian classifier, posterior probabilities, normal distribution, decision boundary.

I. INTRODUCTION

The minimum error rate classifier seeks a decision rule that minimizes the probability of error which is the error rate. This classifier takes decision based on the most posterior probabilities. In this experiment, a test sample points have been given to classify. The likelihood probabilities of a sample is given by the normal distribution. Normal distribution can be define with two parameters: sigma and mean which have been given. The details work procedures has been explained in the methodology section.

As Bayesian classifier works with posterior probabilities the decision rule is as follows:

$$\text{if } P(w_1|x) > P(w_2|x) \text{ Then } x \in w_1$$

$$\text{if } P(w_1|x) < P(w_2|x) \text{ Then } x \in w_2$$

The posterior probabilities can be calculated with the help of likelihood probabilities.

$$P(w_i|x) = P(x|w_i)P(w_i)$$

$$\text{Ln}(P(w_i|x)) = \text{Ln}(P(x|w_i)P(w_i))$$

$$= \text{Ln}P(x|w_i) + \text{Ln}P(w_i)$$

As our data points are 2D so we have to use multivariate normal distribution. For 2D data the normal distribution formula is:

$$g_i(x) = w^T x + w_0$$

$$g_i(x) = \text{Ln}P(x|w_i) + \text{Ln}P(w_i)$$

$$= -\frac{D}{2}\text{Ln}2\pi - \frac{1}{2}\text{Ln}|\Sigma| - \frac{1}{2}(x - \mu_i)^T \Sigma^{-1}(x - \mu_i) + \text{Ln}P(w_i)$$

II. EXPERIMENTAL DESIGN / METHODOLOGY

A. Description of the different tasks:

A set of 2D sample points named test.txt has been given.

Task 1: Classify the sample points from “test.txt”.

Task 2: Classified samples should have different colored markers according to the assigned class label.

Task 3: Draw a figure which should include these points, the corresponding probability distribution function along with its contour.

Task 4: Draw decision boundary.

Given Normal Distribution Formula:

$$N_k(x_i|\mu_k, \Sigma_k) = \frac{1}{\sqrt{(2\pi)^D |\Sigma_k|}} e^{(-\frac{1}{2}(x_i - \mu_k)^T \Sigma^{-1}(x_i - \mu_k))}$$

B. Implementation:

1) Plotting of classified sample data with different marker:

Using normalized formula of normal distribution i calculated the value of $g(x)$ for each data points with the two given Normal distribution and check for the following condition $g_1(x) > g_2(x)$. If the condition is true then the sample point x belongs to the corresponding regions of the normal distributions.

The value of $g_1(x)$ greater than $g_2(x)$ means the sample point likelihood probabilities close to the used normal distribution so we can assign this sample point to that region with a specific color. The output is in Fig 1.

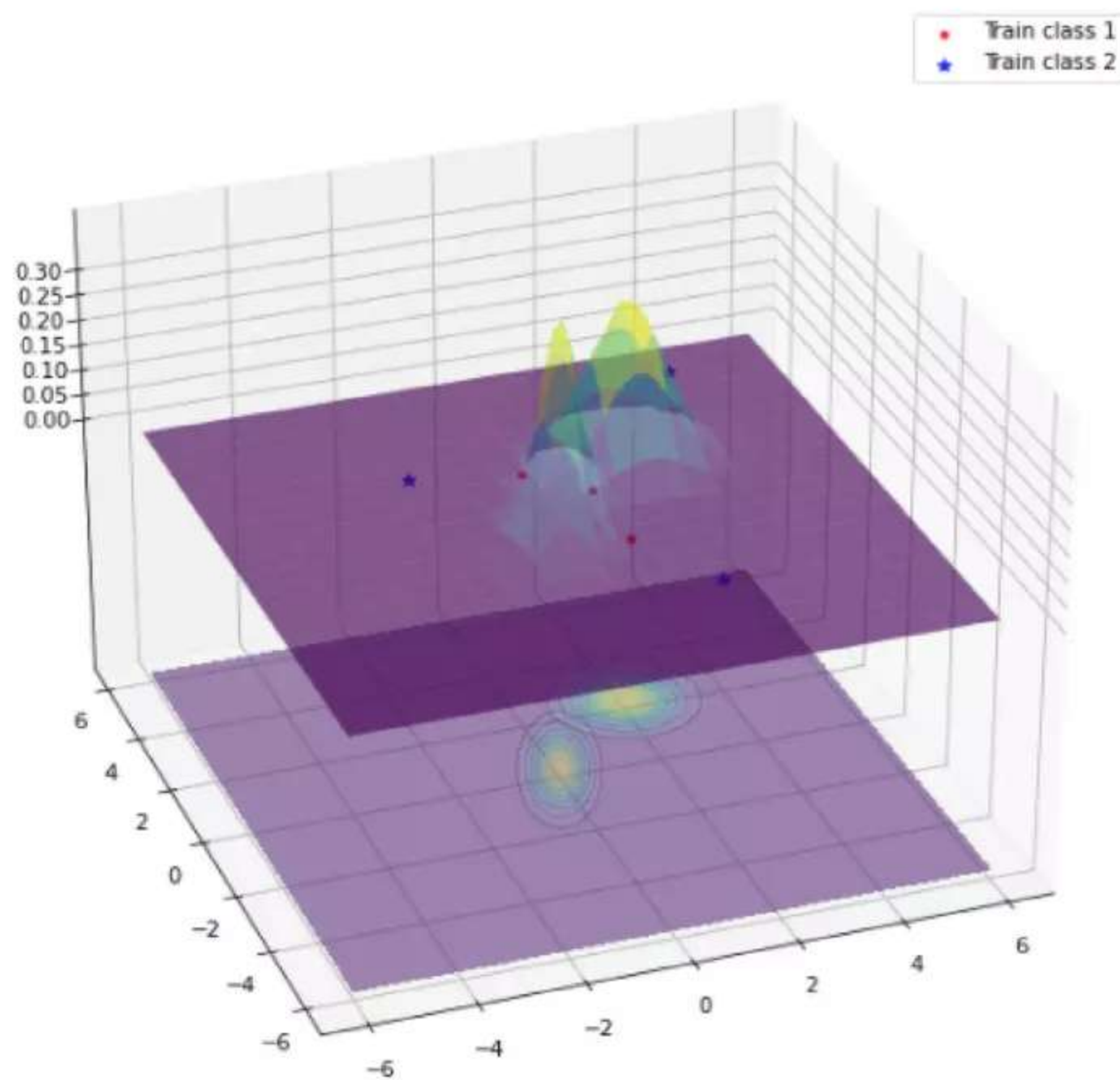


Fig. 1. Classified Sample Point Plotting

2) *Decision Boundary drawing*: To draw the decision boundary we have to obtain the equation of the decision boundary and we know that the decision boundary comes from $g_1(x) - g_2(x) = 0$. Using defined function multivariate normal we calculate the value of decision boundary and then plot the decision boundary. For better visualization i rotate the 3D graph. Fig 2. In this we also plot the surface plot and contour plot using multivariate normal distribution.

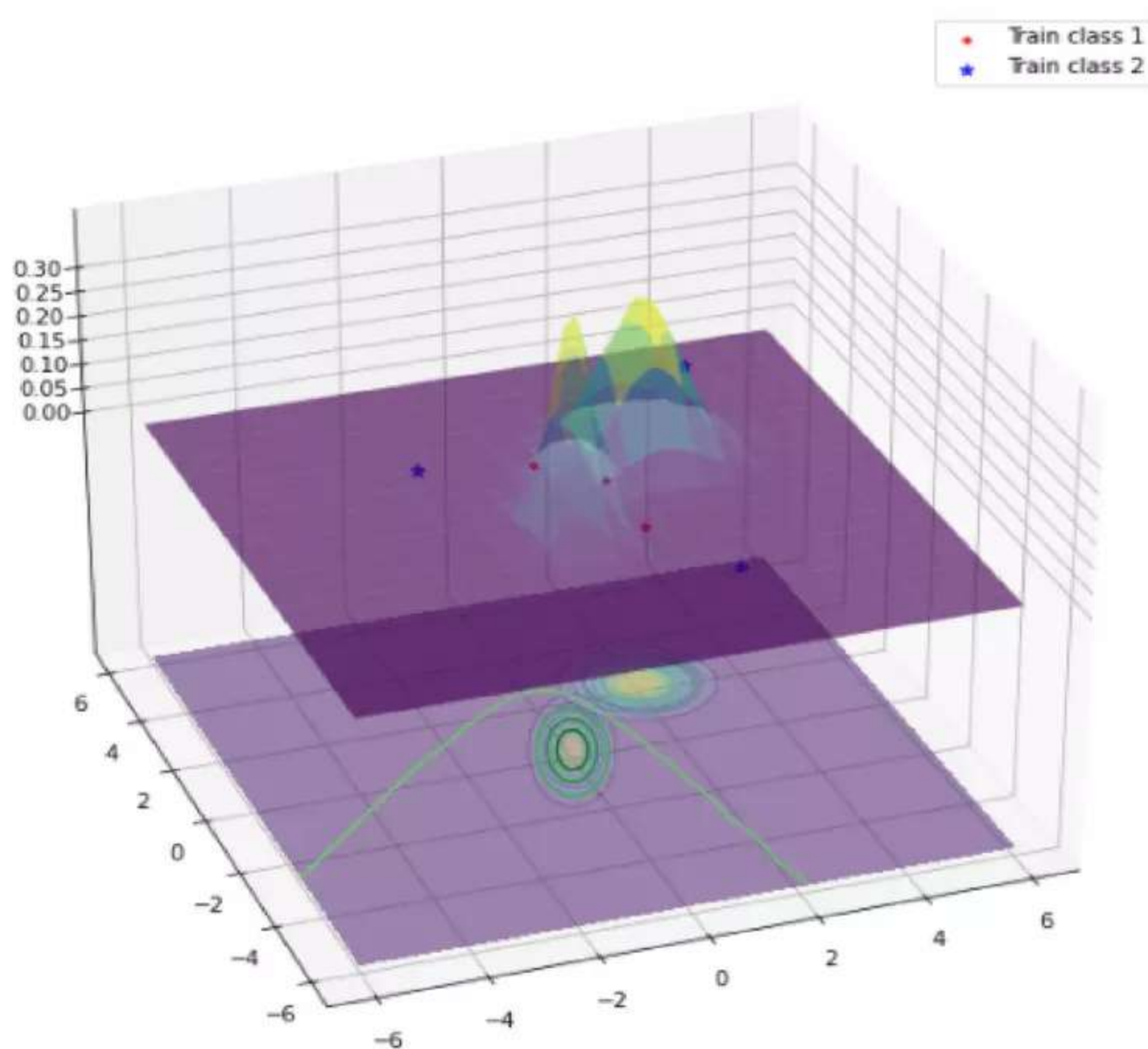


Fig. 2. Decision Boundary

III. RESULT ANALYSIS

Using the given normal distribution formula, i classified the test.txt sample points and found that 3 data points classified

into class 1 and 3 data points classified into class 2. From the 3D graph we can also see that the distribution is parameterized by mu and sigma. The upper plot is surface plot and the lower plot is contour plot with decision boundary.

IV. CONCLUSION

In this experiment we came to know that how a minimum error rate classifier works and what does it mean by sigma and mean of Normal distribution. However there are some limitations of this classifier. This classifier fully depends on probability and the Normal distribution should be known.

V. ALGORITHM IMPLEMENTATION / CODE

```
1 import pandas as pd
2 import numpy as np
3 import random
4 import matplotlib.pyplot as plt
5
6 p_train = pd.read_csv('assignment3.txt', header=None
7 , sep=',', dtype='float64')
8 p_train=np.array(p_train)
9
10 signal = np.array([[.25,.3],[.3,1]])
11 sigma2 = np.array([[.5,0],[0,.5]])
12 mu1 = np.array([0,0])
13 mu2 = np.array([2,2])
14
15 pw1 = 0.5
16 pw2 = 0.5
17
18 x=len(p_train)
19 print(x)
20
21 classs = []
22 for i in range(x):
23     g1 = -0.5*(np.dot(np.dot(p_train[i]-mu1,np.
24         linalg.inv(signal)), (p_train[i]-mu1).T)) - np.
25         log(2*np.pi) - (0.5 * np.log(np.linalg.det(
26             signal)))+pw1
27     g2 = -0.5*(np.dot(np.dot(p_train[i]-mu2,np.
28         linalg.inv(sigma2)), (p_train[i]-mu2).T)) - np.
29         log(2*np.pi) - (0.5 * np.log(np.linalg.det(
30             sigma2)))+pw2
31     if g1>g2:
32         temp = []
33         temp.append(p_train[i][0])
34         temp.append(p_train[i][1])
35         temp.append(1)
36         classs.append(temp)
37     else:
38         temp = []
39         temp.append(p_train[i][0])
40         temp.append(p_train[i][1])
41         temp.append(2)
42         classs.append(temp)
43
44 x1=[]
45 y1=[]
46 x2=[]
47 y2=[]
48
49 for i in range(x):
50     if classs[i][2]==1:
51         x1.append(classs[i][0])
52         y1.append(classs[i][1])
53     else:
54         x2.append(classs[i][0])
55         y2.append(classs[i][1])
```



```

51
52
53 print(x1,y1)
54 print(x2,y2)
55
56
57 import numpy as np
58 import matplotlib.pyplot as plt
59 from matplotlib import cm
60 from mpl_toolkits.mplot3d import Axes3D
61
62
63 N = 60
64 X = np.linspace(-6, 6, N)
65 Y = np.linspace(-6, 6, N)
66 X, Y = np.meshgrid(X, Y)
67
68
69 pos = np.empty(X.shape + (2,))
70 pos[:, :, 0] = X
71 pos[:, :, 1] = Y
72
73 def multivariate_normal(pos, mu, Sigma):
74
75     n = mu.shape[0]
76     Sigma_det = np.linalg.det(Sigma)
77     Sigma_inv = np.linalg.inv(Sigma)
78     N = np.sqrt((2*np.pi)**n * Sigma_det)
79     fac = np.einsum('...k,kl,...l->...', pos-mu,
80                     Sigma_inv, pos-mu)
81
82     return np.exp(-fac / 2) / N
83
84 Z1 = multivariate_normal(pos, mu1, sigma1)
85 Z2 = multivariate_normal(pos, mu2, sigma2)
86 db = Z1 - Z2
87
88
89
90 fig = plt.figure(figsize=(15,10))
91 ax = fig.gca(projection='3d')
92
93 ax.plot_surface(X, Y, Z1, rstride=3, cstride=3,
94                 linewidth=0,alpha = 0.4, antialiased=True,cmap='
95                 viridis')
96 ax.plot_surface(X, Y, Z2, rstride=3, cstride=3,
97                 linewidth=0,alpha = 0.4, antialiased=True,cmap='
98                 viridis')
99
100 cset1 = ax.contourf(X, Y, Z1, zdir='z', offset=-0.5,
101                    alpha = 0.3, cmap='viridis')
102 cset2 = ax.contourf(X, Y, Z2, zdir='z', offset=-0.5,
103                    alpha = 0.3, cmap='viridis')
104
105 db2 = ax.contour(X, Y, db, zdir='z', offset=-0.5,
106                alpha = 1, cmap='Greens')
107
108 # Adjust the limits, ticks and view angle
109 ax.set_zlim(-0.5,0.4)
110 ax.set_zticks(np.linspace(0,0.3,7))
111 ax.view_init(30, -110)
112 ax.scatter(x1,y1,color='r',marker='.',alpha=0.8, s
113            =40, label='Train class 1')
114 ax.scatter(x2,y2,color='b',marker='*',alpha=0.8, s
115            =40, label='Train class 2')
116
117
118 ax.legend()
119 plt.show()

```

- [2] G. Eason, B. Noble, and I. N. Sneddon, “On certain integrals of Lipschitz-Hankel type involving products of Bessel functions,” Phil. Trans. Roy. Soc. London, vol. A247, pp. 529–551, April 1955.

REFERENCES

- [1] Data Visualization: Visualizing the bivariate Gaussian distribution